Lab 7: Fuzzing for Input Validation Bugs

Name	Aryaman D K
SRN	PES1UG24AM344
Assigned repo	L6_L7_39_password_policy (lab7)
Application	password_policy.py - Password Policy Checker
Tools	Python 3, Hypothesis 6, pytest

Objective

To use property-based fuzz testing with the Hypothesis library to find input-validation bugs in **password_policy.py**, and then harden the code so that it handles every unexpected input safely.

Functions under test

first_character_type(pw) - classifies the first character as digit / letter / symbol.
split_credentials(text) - splits a *user:password* string into a (user, password) tuple.
digit_ratio(pw) - returns the fraction of characters in the password that are digits.

Step 1: Installing dependencies

`pip install hypothesis` and `pip install pytest` were run to install the fuzzing library and the test runner.

Step 2: Running the buggy application

Command: `python processor.py` (named `password_policy.py` in this repo). With normal input the program works, but empty or badly formatted input makes it crash.

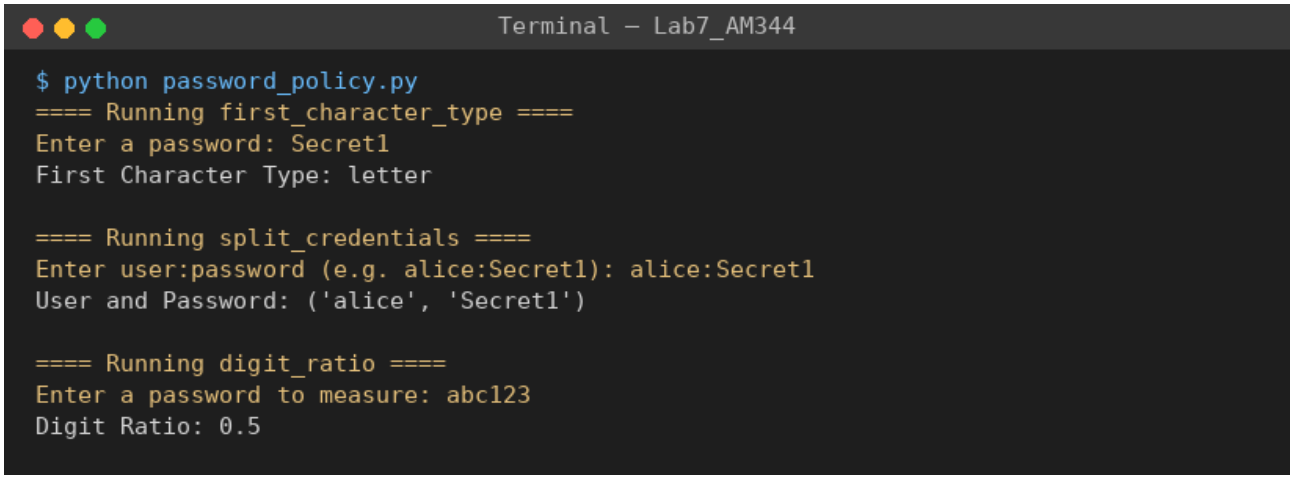


Fig 2.1 - Buggy application with valid input

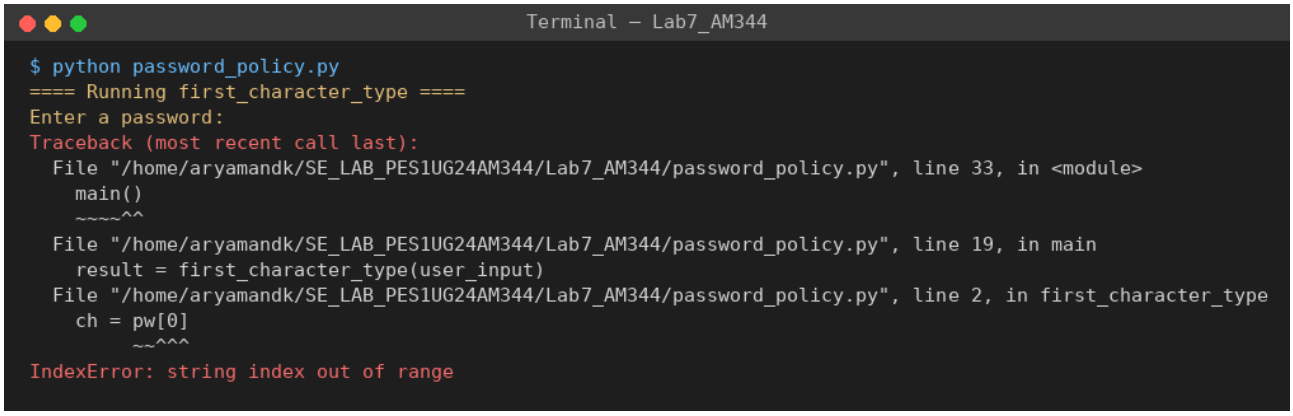


Fig 2.2 - Buggy application crashes on an empty password (IndexError)

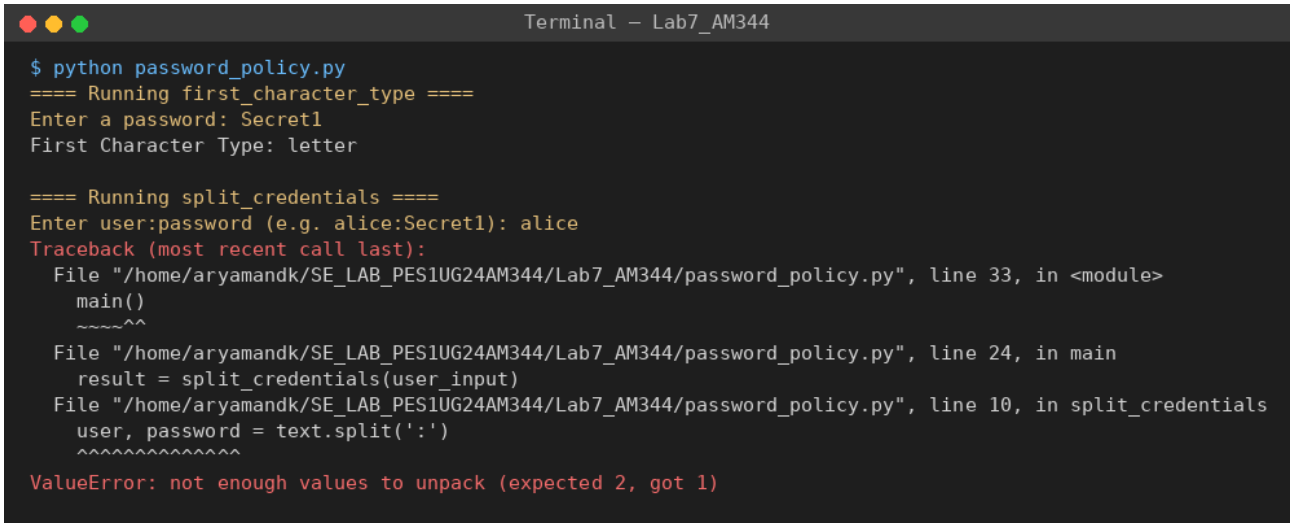


Fig 2.3 - Buggy application crashes when the ':' separator is missing (ValueError)

Step 3: Completing test_password_policy.py with Hypothesis

The three starter tests were completed so that Hypothesis feeds random Unicode strings and **None** to each function (`st.text() | st.none()`) and asserts that no exception is raised. Three extra property tests were added to check that the outputs are also correct:

Test	Property checked
`test_first_character_type_no_crash`	never raises for any text or None
`test_first_character_type_valid_label`	non-empty text always gives digit / letter / symbol
`test_split_credentials_no_crash`	never raises for any text or None
`test_split_credentials_round_trip`	'user:password' splits back correctly, even if password has ':'
`test_digit_ratio_no_crash`	never raises for any text or None
`test_digit_ratio_in_range`	result is always between 0.0 and 1.0

Step 4: Running the fuzz tests on the buggy code

Command: `python -m pytest test_password_policy.py`. Result: **5 failed, 1 passed**. Hypothesis automatically shrank each failure to the smallest failing input. The full output is shown across four screenshots.

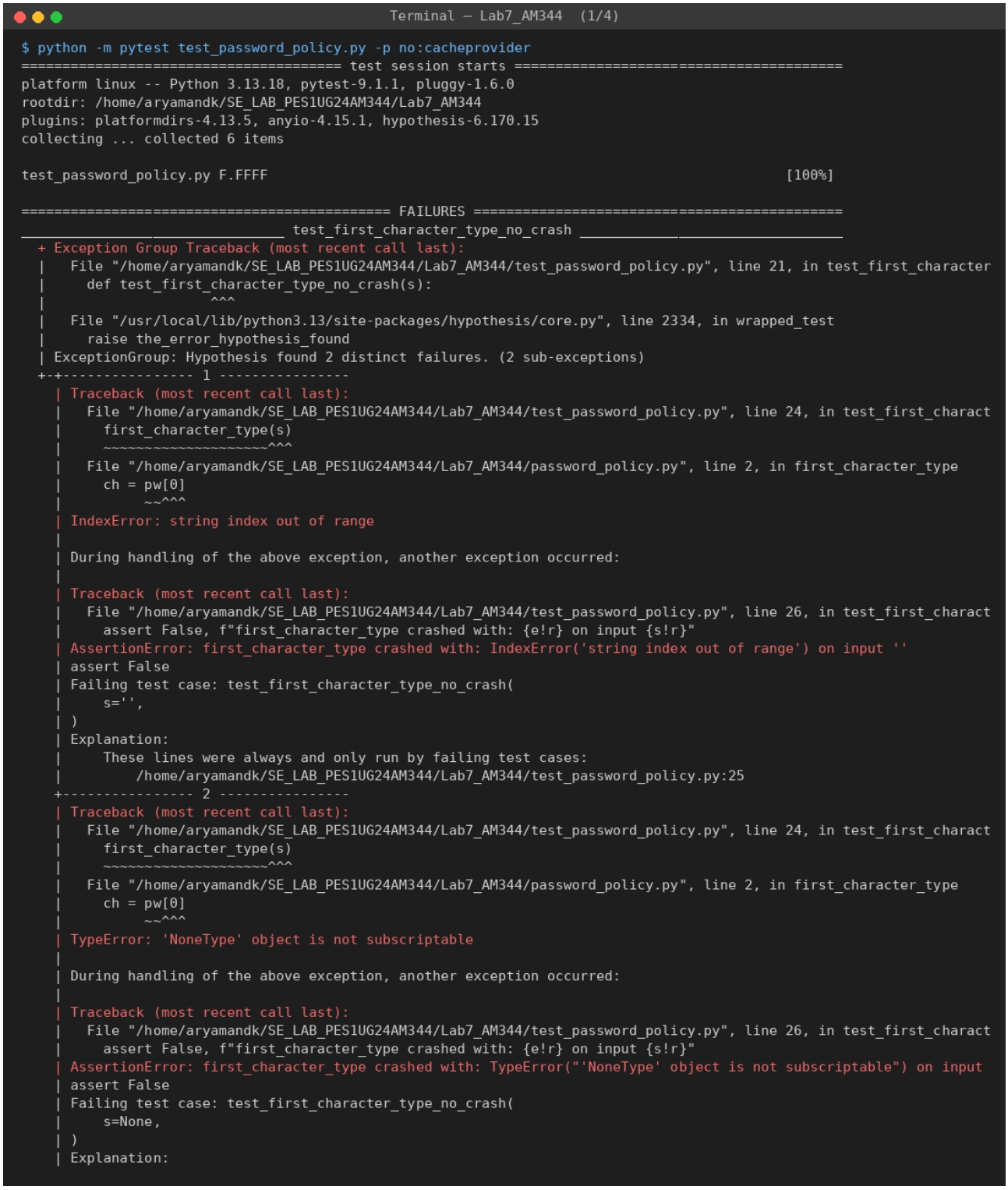


Fig 4.1 - pytest output on buggy code (1/4)

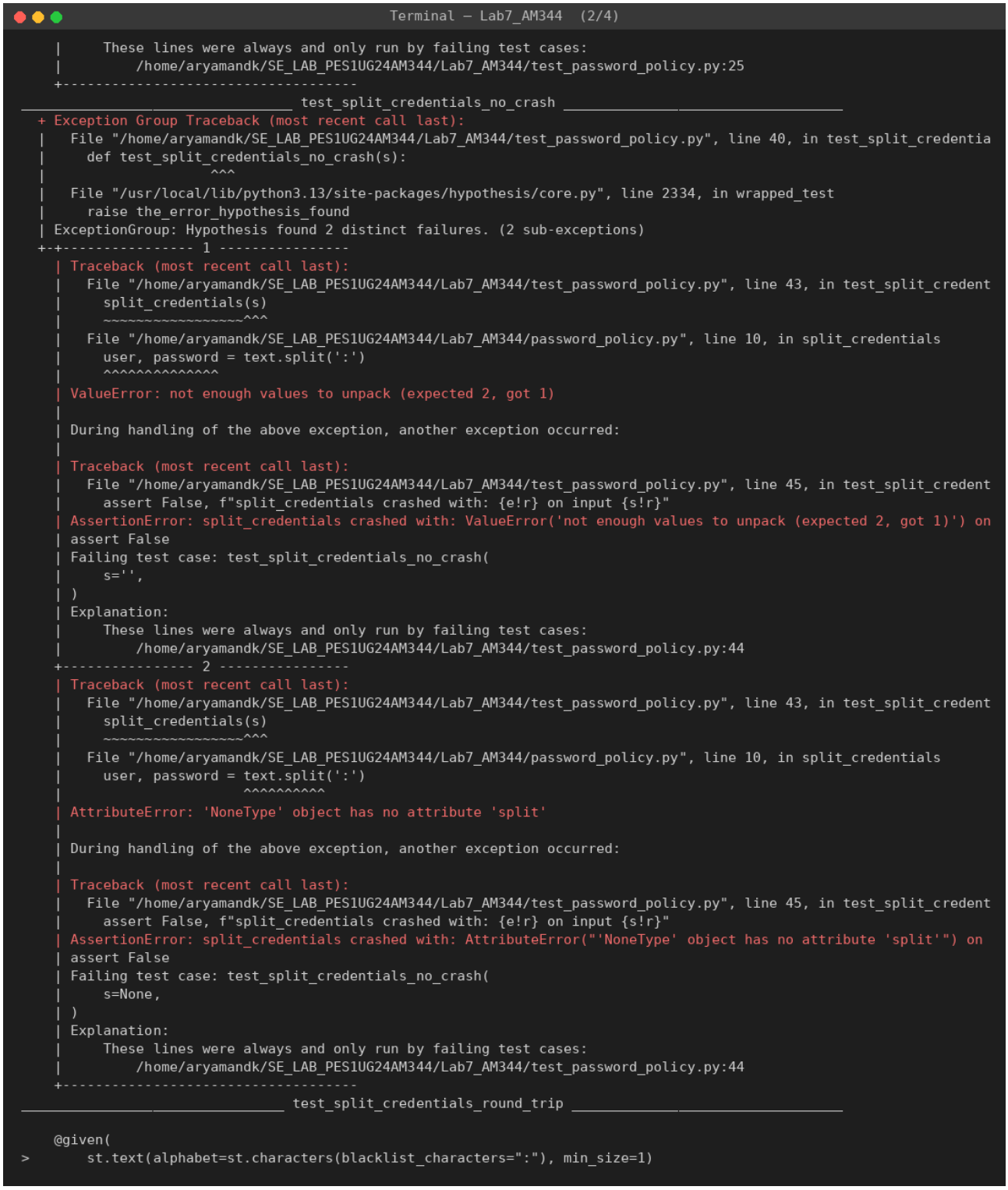


Fig 4.2 - pytest output on buggy code (2/4)

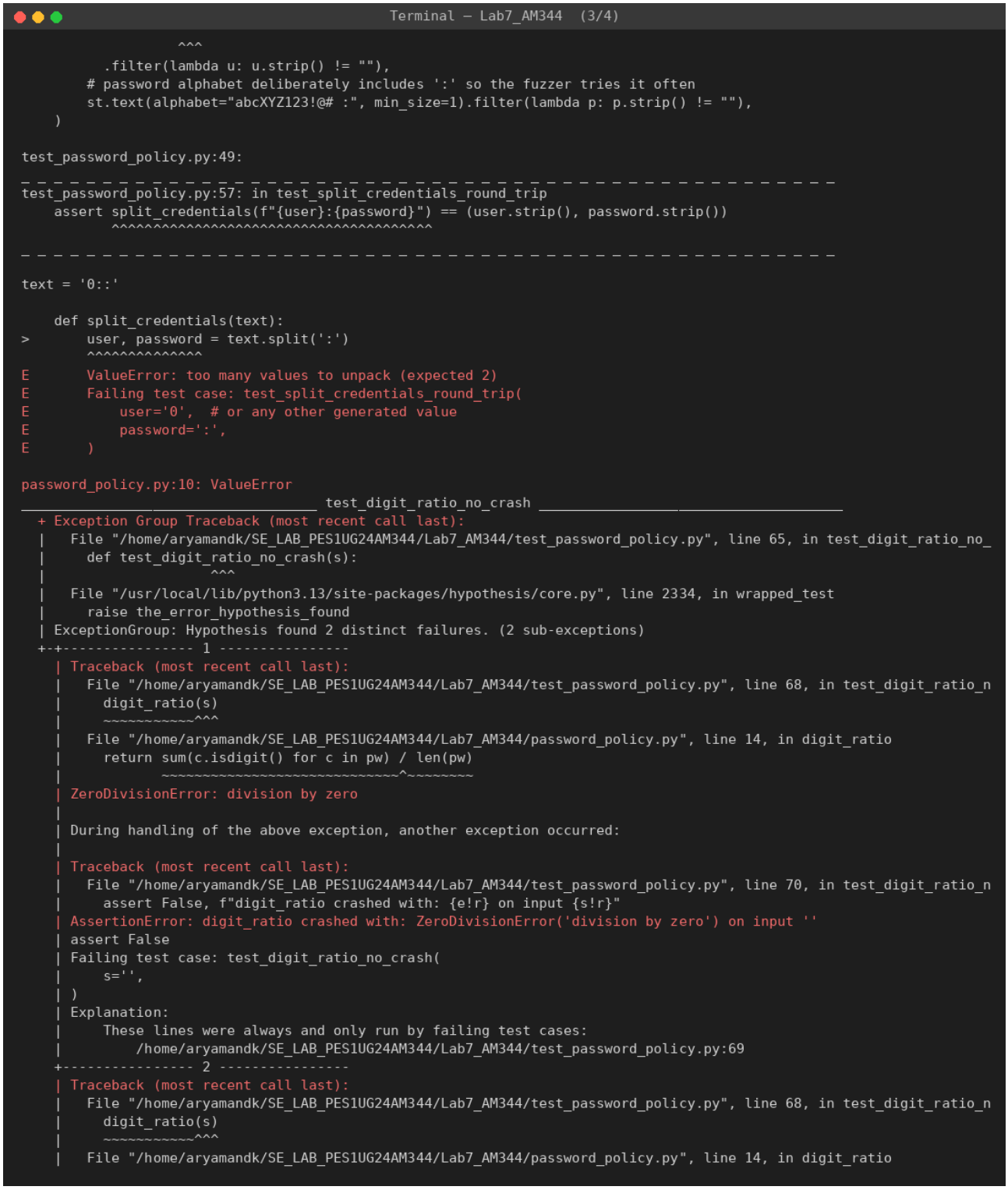


Fig 4.3 - pytest output on buggy code (3/4)

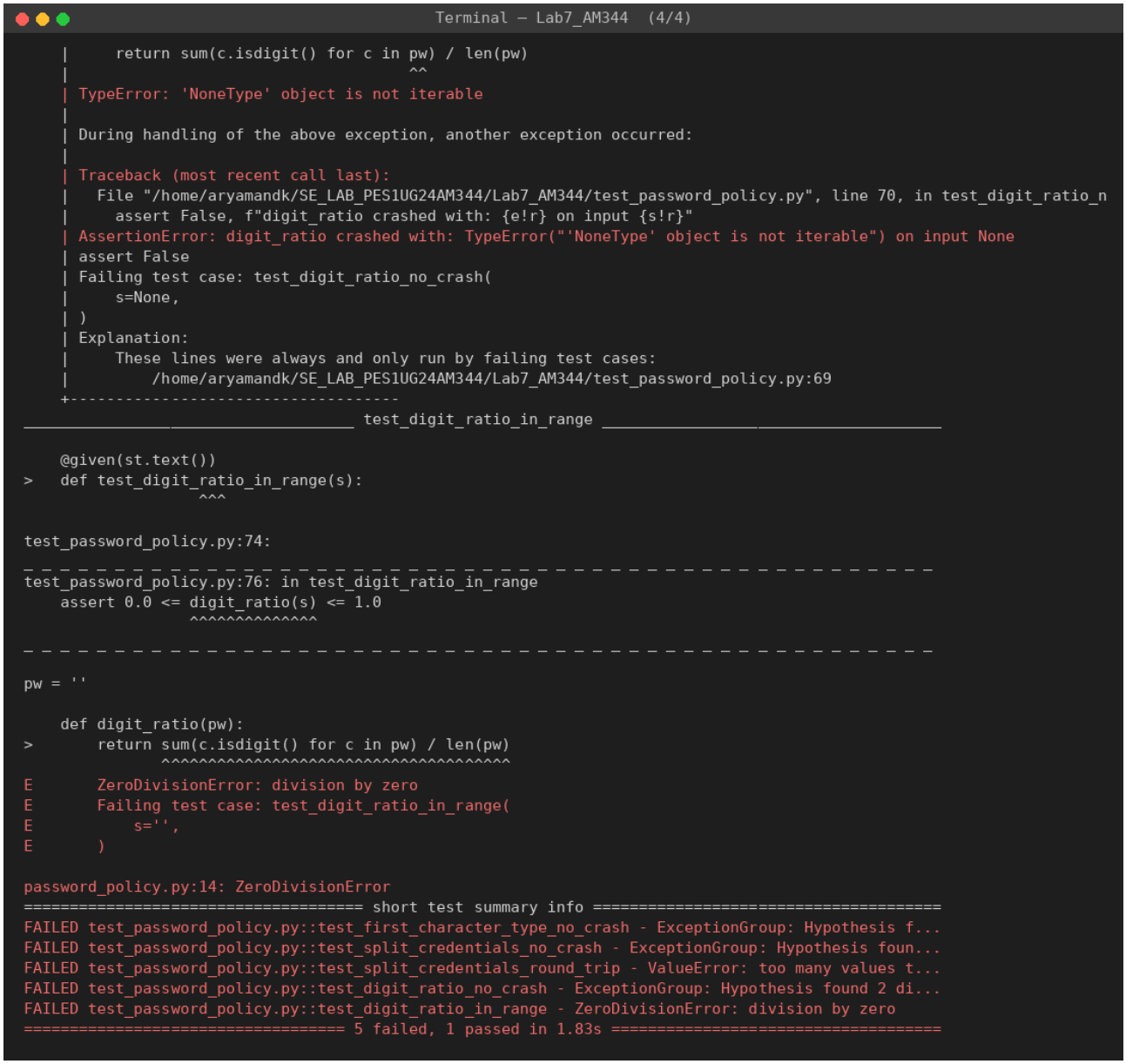


Fig 4.4 - pytest output on buggy code (4/4)

Bugs found by fuzzing

#	Function	Failing input	Error	Fix applied
1	first_character_type	''	IndexError	return 'empty'
2	first_character_type	None	TypeError	return 'invalid'
3	split_credentials	'' / 'alice'	ValueError	return None if ':' missing
4	split_credentials	None	AttributeError	return None for non-string
5	split_credentials	'0::'	ValueError (too many)	split(':', 1)
6	split_credentials	':' / 'user:'	returns empty parts	return None if a part is empty
7	digit_ratio	''	ZeroDivisionError	return 0.0
8	digit_ratio	None	TypeError	return 0.0

Bug 6 was not a crash, but returning an empty username or password is invalid output, so it was hardened at the same time.

Step 5: Fixing the application

Each function now checks that the input is a string and is not empty before using it. **split_credentials** splits only on the first ':' so passwords containing ':' work, and returns **None** for badly formatted input. **main()** prints a clear error message instead of crashing.

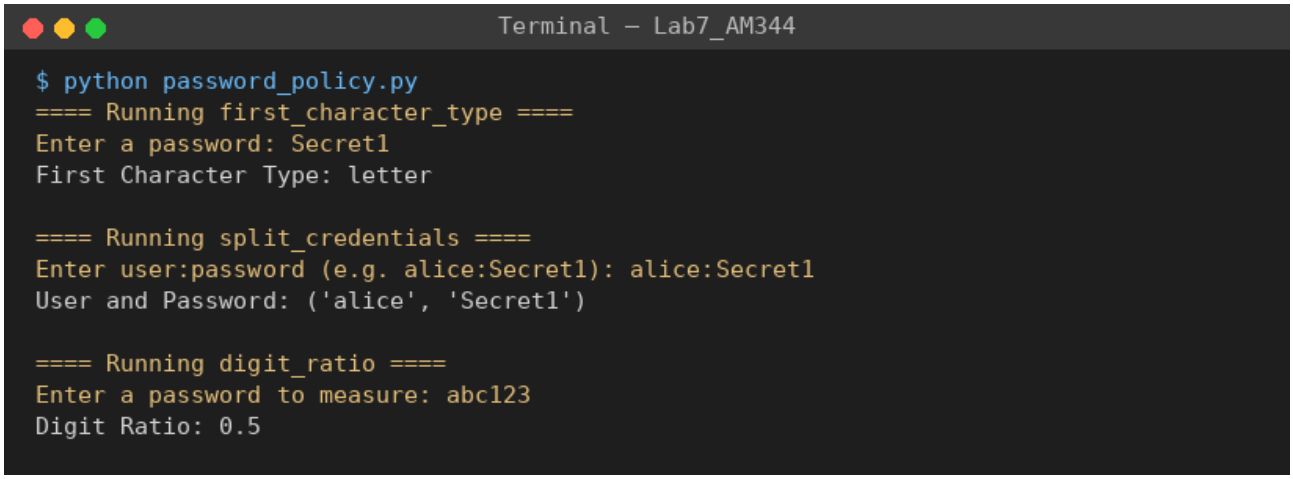


Fig 5.1 - Fixed application with valid input

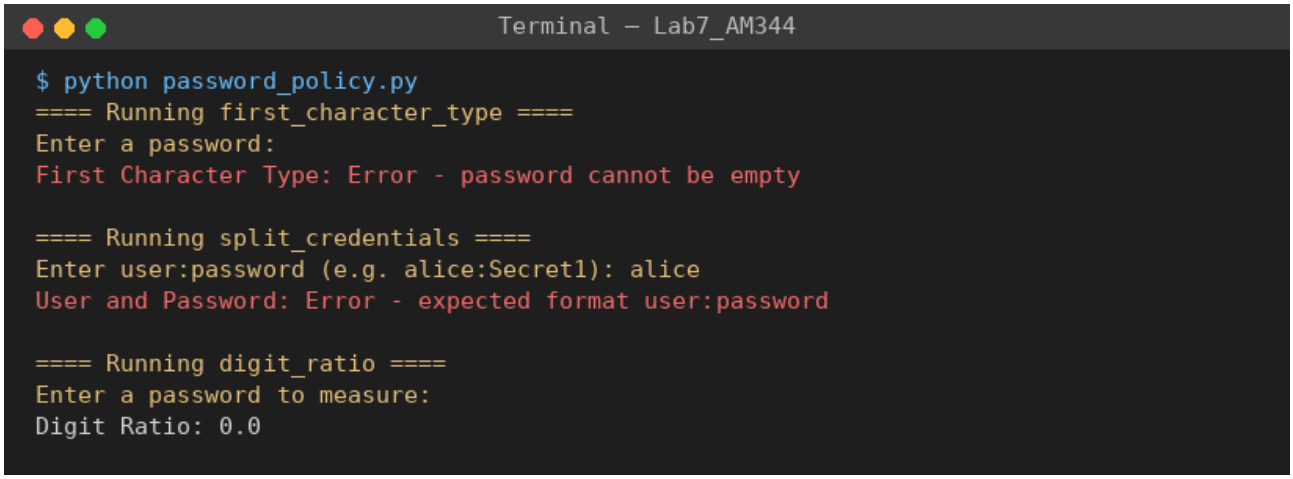


Fig 5.2 - Fixed application handles empty input and a missing ':' without crashing

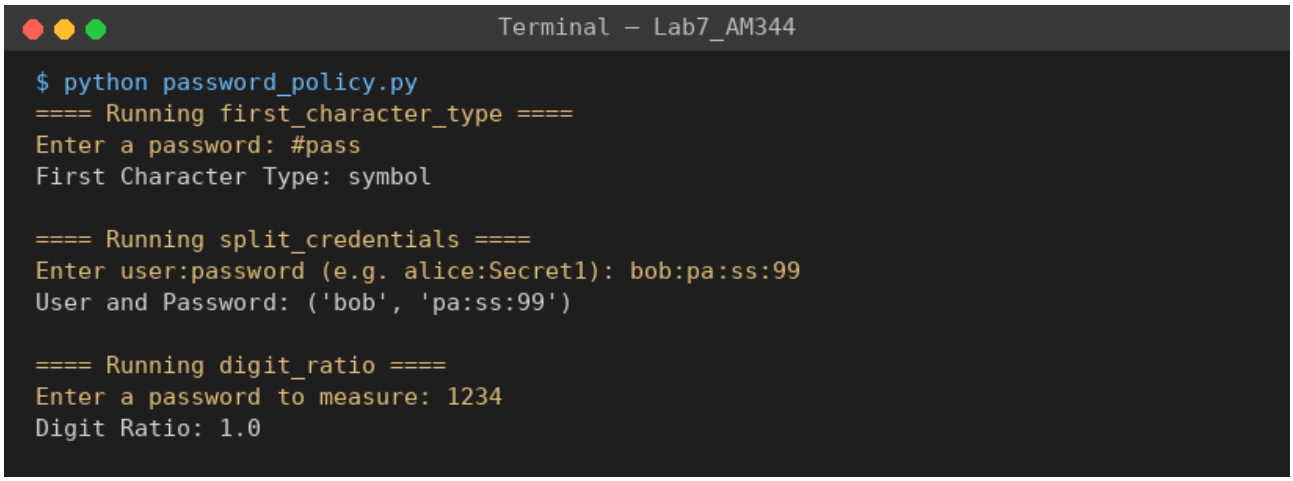


Fig 5.3 - Fixed application handles a symbol first character and a password containing ':'

Step 6: Re-running the fuzz tests on the fixed code

Command: `python -m pytest test_password_policy.py -v`. Result: **all 6 tests passed**.

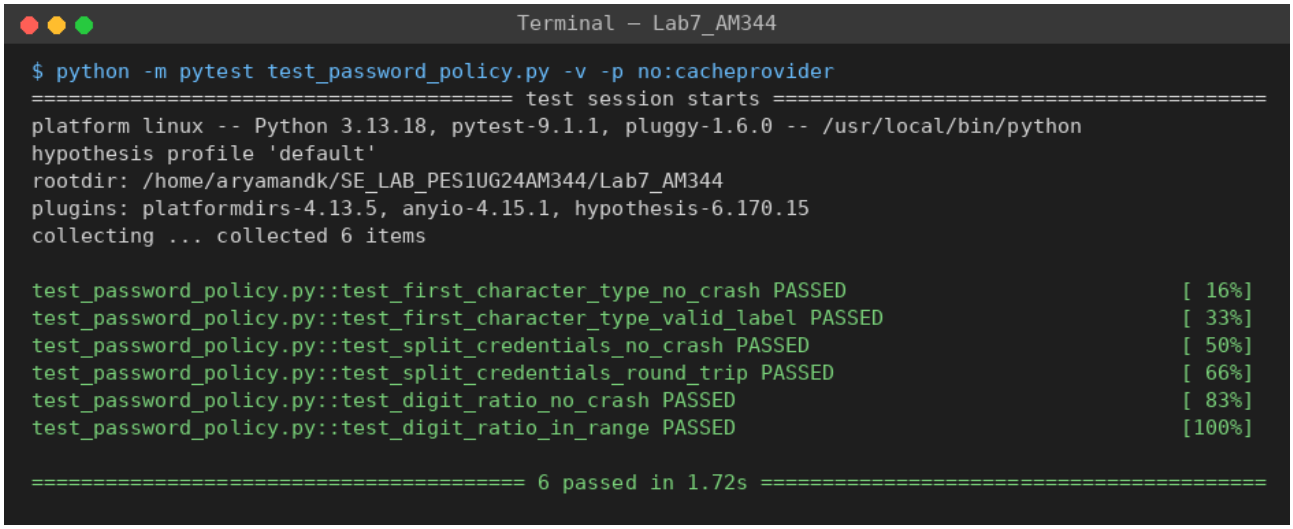


Fig 6.1 - All fuzz tests pass on the fixed application

Conclusion

Hand-written tests with normal values did not reveal any problems, but Hypothesis found 8 input-validation bugs within seconds by trying empty strings, None and unusual separators. After adding defensive checks, every function handles unexpected input safely and all fuzz tests pass.